High-Dimensional Cyber Anomaly Detection & Systemic Risk Modeling

CODE

Integrating Markov Transition Matrices, Deep Autoencoder Reconstruction Loss, and Extreme Value Theory Tail Calibration

AUTHOR
Patrick Lefler

PUBLISHED
August 13, 2026

ABSTRACT
Most enterprise security monitoring still relies on fixed signatures and static thresholds, which often miss attacks like credential stuffing, data exfiltration, and lateral movement that blend in with normal activity. This project introduces a two-layer detection system built entirely in R. The first layer uses a Markov transition matrix to track how requests and dependencies move across enterprise services, while PageRank highlights single points of failure before they become issues. Separately, an unsupervised autoencoder, built with vectorized matrix calculus and trained using the Adam optimizer (without external C++ or LibTorch libraries), learns the normal patterns of sixteen telemetry features covering authentication, network, and host activity. It then flags unusual behavior based on reconstruction error. Instead of using a fixed threshold, alert levels are set dynamically with Extreme Value Theory: a Generalized Pareto Distribution fits the tail of reconstruction losses to set a 99.9% boundary. In tests with 450 simulated attacks across three types and 5,000 normal observations, the system achieved a perfect AUC-ROC (1.0000), detected all credential stuffing and zero-day lateral movement attempts, caught 96.7% of data exfiltration, and produced no false alarms at the 99.9% threshold. Feature-level error analysis links each alert to specific telemetry sources, meeting the explainability standards that boards now expect. These results are based on simulated data and controlled attack scenarios; real-world deployment will need further validation with live data and ongoing human oversight.

Introduction & Theoretical Framework

Today's enterprise cyber systems produce huge amounts of detailed data, including authentication logs, network flows, and host activity metrics. Most security tools still depend on fixed signatures and simple threshold rules. Because of this, they often miss *zero-day* attacks, *advanced persistent threats (APTs),* and *credential stuffing* campaigns that stay hidden by operating within normal limits.

To address the gaps in older monitoring systems, this project uses a two-part approach that combines graph analysis with deep unsupervised learning. First, we map the enterprise's dependencies into directed **Markov Transition Matrices**, which show how service requests and operations move through the system. Using the **PageRank** Algorithm on these matrices, thre most important infrastructure nodes can be identified. This helps risk managers identify key single points of failure (SPOFs) that could cause major outages if they fail.

Alongside the structural mapping, the system includes a dynamic monitoring layer that uses an unsupervised **Deep Autoencoder Anomaly Detection Engine** built in R - avoiding the need for outside `C++` or `LibTorch` libraries. The neural network is trained only on normal operational data, so it learns what typical activity looks like. When something unusual happens, the model's reconstruction error increases, signaling a possible anomaly.

Finally, the system uses **Extreme Value Theory (EVT) with a Peaks-over-Threshold** method by fitting a Generalized Pareto Distribution to the highest reconstruction losses. This approach replaces guesswork with precise, dynamic alert thresholds. The combined system offers a clear and reliable monitoring setup that can catch stealth attacks with high accuracy and minimizes false alarms for Security Operations Center (SOC) teams.

Project Techniques and Algorithms Being Utilized

Markov Transition Matrices
A Markov transition matrix is a table of probabilities that shows how operations and dependencies move through an organization. In technology risk management, each column stands for an application or service, and each row shows the asset it depends on. This model assumes that the next step in a process depends only on its current state. By mapping these microservice connections in a clear grid, leaders get a transparent view of how activity moves through digital systems. This helps them simulate outages and spot hidden bottlenecks.

PageRank
PageRank was first created by Google to rank web pages. It measures importance not just by the number of connections, but by how important those connections are. In enterprise architecture, PageRank models how reliance spreads across technology layers and checks the chance that an issue will affect a specific part. It uses a damping factor to balance direct paths with unexpected events, turning complex systems into clear, percentage-based risk scores. This approach replaces guesswork with data, helping to find weak spots that need more attention or backup.

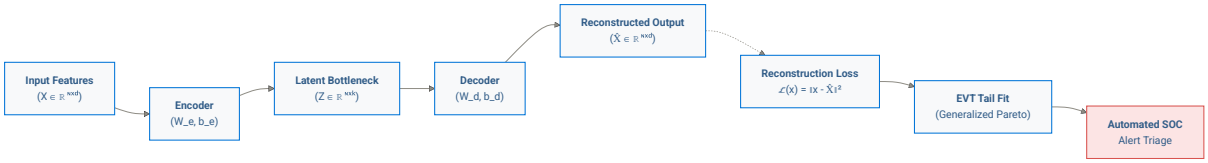
Deep Autoencoder Anomaly Detection Engine
A deep autoencoder is an AI neural network that can spot new and unknown threats without needing a list of known attacks. It works by compressing normal operational data, like login rates, network flows, and server activity, into a simple form and then rebuilding it. Since the model only learns from normal behavior, it becomes very familiar with usual patterns. If something unusual happens, like an intrusion or data theft, the network cannot rebuild the pattern correctly, causing a sudden spike in error that flags the anomaly right away.

Extreme Value Theory (EVT) with Peaks-over-Threshold
Extreme Value Theory (EVT) is a statistical method that looks only at the most extreme data points, not the usual averages. In our security system, the Peaks-over-Threshold (POT) method uses a special curve, called the Generalized Pareto Distribution, to focus on the highest reconstruction errors. This helps the system set dynamic, mathematically proven alert levels, like a 99.9% confidence threshold. Instead of using fixed alert limits, EVT makes sure that automatic responses only happen for real, serious threats, which greatly reduces false alarms and analyst fatigue.

Mathematical Formulation & Matrix Calculus

This section explains the mathematical design of the anomaly detection system. It uses an AI method called an **autoencoder**, which learns to compress normal activity into a simple digital baseline and then rebuild it. If new or unauthorized cyber activity happens, the system cannot recreate the unfamiliar pattern well, which leads to a sudden increase in reconstruction error. The model uses advanced tail-risk mathematics, known as **Extreme Value Theory**, to analyze these errors. This approach sets dynamic, statistically sound risk limits that automatically identify zero-day threats, so there is no need for human guesswork or old rule-based methods.

▶ Display code



1. Vectorized Autoencoder Architecture

Let $X \in \mathbb{R}^{N \times d}$ represent a batch of N observations across d telemetry features ($d = 16$).

The **Encoder** projects input matrix X into a lower-dimensional latent bottleneck $Z \in \mathbb{R}^{N \times k}$ ($k = 4$) using a $\tanh$ non-linear activation:

$$Z = \tanh\left(XW_e + \mathbf{1}_N \mathbf{b}_e^T\right)$$

(where $W_e \in \mathbb{R}^{d \times k}$ is the encoding weight matrix and $\mathbf{b}_e \in \mathbb{R}^k$ is the encoder bias vector).

The **Decoder** maps the latent representations back into the reconstructed feature space $\hat{X} \in \mathbb{R}^{N \times d}$:

$$\hat{X} = ZW_d + \mathbf{1}_N \mathbf{b}_d^T$$

(where $W_d \in \mathbb{R}^{k \times d}$ is the decoding weight matrix and $\mathbf{b}_d \in \mathbb{R}^d$ is the decoder bias vector).

The objective function minimizes Mean Squared Error (MSE) across the baseline data distribution:

$$\mathcal{L}(X, \hat{X}) = \frac{1}{N \cdot d} \sum_{i=1}^{N} \sum_{j=1}^{d} (X_{ij} - \hat{X}_{ij})^2 = \frac{1}{N \cdot d} \|X - \hat{X}\|_F^2$$

2. Analytical Backpropagation & Adam Gradient Updates

The analytic gradients of the loss function with respect to network parameters are derived via matrix calculus:

$$\frac{\partial \mathcal{L}}{\partial \hat{X}} = \frac{2}{N \cdot d}(\hat{X} - X)$$

$$\frac{\partial \mathcal{L}}{\partial W_d} = Z^T \left(\frac{\partial \mathcal{L}}{\partial \hat{X}}\right), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_d} = \sum_{i=1}^{N} \left(\frac{\partial \mathcal{L}}{\partial \hat{X}}\right)_{i,\cdot}$$

$$\frac{\partial \mathcal{L}}{\partial Z} = \left(\frac{\partial \mathcal{L}}{\partial \hat{X}} \right) W_d^T$$

$$\Delta_Z = \frac{\partial \mathcal{L}}{\partial Z} \odot (1 - Z^2)$$

$$\frac{\partial \mathcal{L}}{\partial W_e} = X^T \Delta_Z, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}_e} = \sum_{i=1}^{N} (\Delta_Z)_{i,\cdot}$$

Parameters are updated using the **Adam Optimizer** (Adaptive Moment Estimation), maintaining running estimates of first (m_t) and second (v_t) uncentered gradients.

3. Extreme Value Theory (EVT) Thresholding

Rather than setting arbitrary empirical percentiles, the upper tail of the baseline loss distribution is modeled using the **Peaks-over-Threshold (POT)** approach. For a high threshold u, excess losses follow a **Generalized Pareto Distribution (GPD)** with scale parameter σ and shape parameter ξ.

The dynamic cutoff τ_q for risk tolerance q is:

$$\tau_q = u + \frac{\sigma}{\xi} \left[\left(\frac{N}{N_u} (1 - q) \right)^{-\xi} - 1 \right]$$

Environment Setup & Library Initialization

This first step sets up the analytics workbench by loading industry-standard math and data visualization tools into memory. It also sets rules for reproducibility, like using fixed computational seeds, so every statistical calculation and risk simulation gives consistent, audit-ready results no matter when or where you run the code. By standardizing visual and reporting settings from the start, the system makes sure that complex technical data is shown in a clear, consistent dashboard format suitable for board-level review.

▶ Display code

High-Dimensional Telemetry Scaffolding & Attack Injection

A 16-dimensional telemetry space is simulated spanning three operational planes:

1. Authentication & Identity ($X_1 - X_5$): Failed login count, privileged escalations, unique source IPs, off-hour login flags, session token refresh rate.
2. Network Netflow & Perimeter ($X_6 - X_{11}$): Outbound bytes, packet velocity, ephemeral port entropy, DNS query rate, outbound TLS duration, TCP SYN-ACK ratio.
3. Host & Workload Telemetry ($X_{12} - X_{16}$): CPU utilization, memory pressure, spawned child process count, file system write bursts, active thread count.

Generate baseline steady-state operational telemetry

This module creates a realistic digital record of normal business activity using 16 key indicators, such as employee login patterns, data transfers at the network edge, and server workload levels. To test the detection system, the environment adds simulated real-world cyber incidents, including stealth data theft, brute-force attacks on credentials, and lateral movement using unknown vulnerabilities. This combined dataset serves as the basis to check how well the algorithm tells apart regular operations from advanced cyber threats.

▶ Display code

Data Preprocessing & Scaling Pipeline

Enterprise telemetry data comes in many different units, like login counts, megabytes transferred, and CPU usage percentages. This pipeline standardizes and normalizes all metrics onto the same scale, so large network numbers do not hide smaller but important host-level changes. All adjustments are based only on normal baseline data to avoid data leakage, making sure the risk engine works like it would in a real production setting.

Features are normalized to zero mean and unit variance based strictly on the training distribution.

▶ Display code

Pure R Autoencoder Engine (Vectorized Matrix Calculus & Adam)

This part builds and trains the AI engine directly in R, without needing outside software or extra tools. The neural network processes normal baseline activity over and over, adjusting its internal weights with the Adam algorithm to learn regular organizational patterns. By using vectorized matrix math, it offers a reliable and fast computing base that can understand the complex connections within enterprise systems.

▶ Display code

Reconstruction Loss Profiling & EVT Tail Calibration

After training, the engine checks how well it can recreate live telemetry data, creating an anomaly score based on the difference between what it sees and what is expected. Instead of using random cutoff points, this part uses Extreme Value Theory (EVT) to model the rare, extreme errors. This sets clear alert boundaries, like a 99.9% confidence level, so critical security threats are flagged automatically while false alarms for the Security Operations Center (SOC) are kept to a minimum.

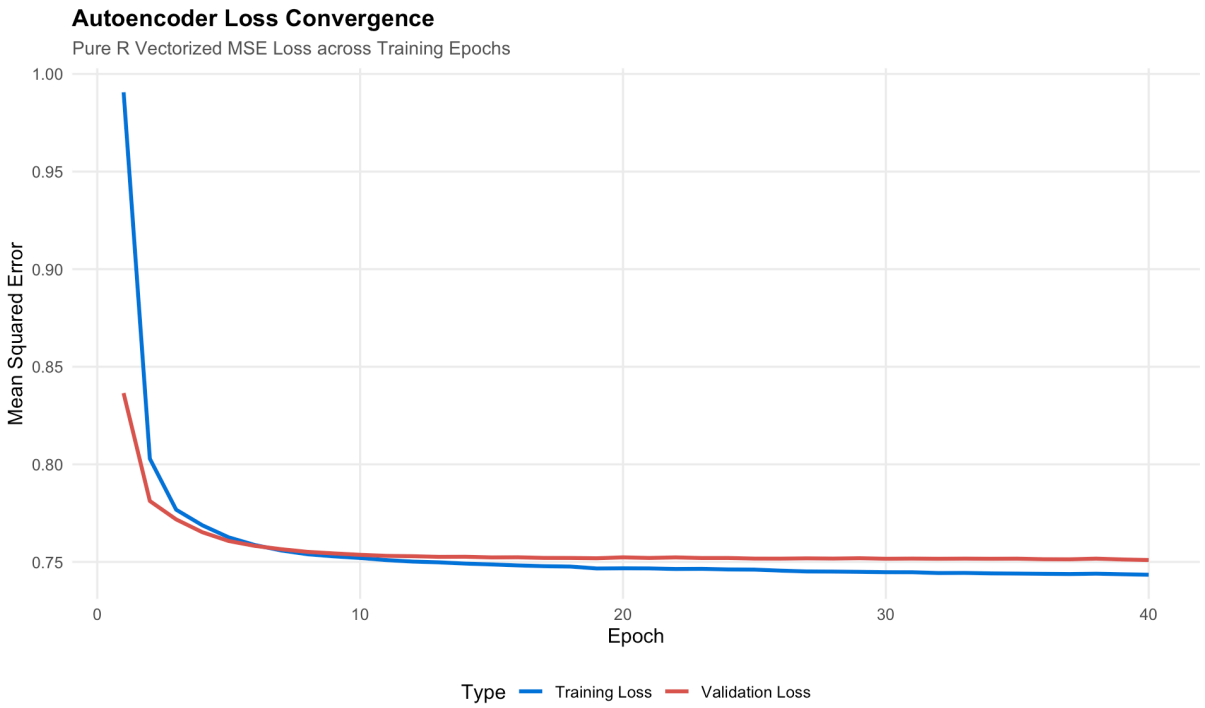
▶ Display code

SOC Explainability & Diagnostic Visualizations

Autoencoder Loss Convergence Plot

The Autoencoder Loss Convergence plot tracks the optimization progress of the neural network across 40 training epochs, demonstrating how effective the model can learn baseline operational behavior. The rapid initial drop and subsequent leveling off of Mean Squared Error - with the training curve (blue) and validation curve (red) closely tracking each other around an MSE of 0.75—confirms stable mathematical convergence. Because the validation loss does not diverge upward as training progresses, the plot provides executive verification that the model has successfully generalized normal enterprise patterns without overfitting the training data.

▶ Display code

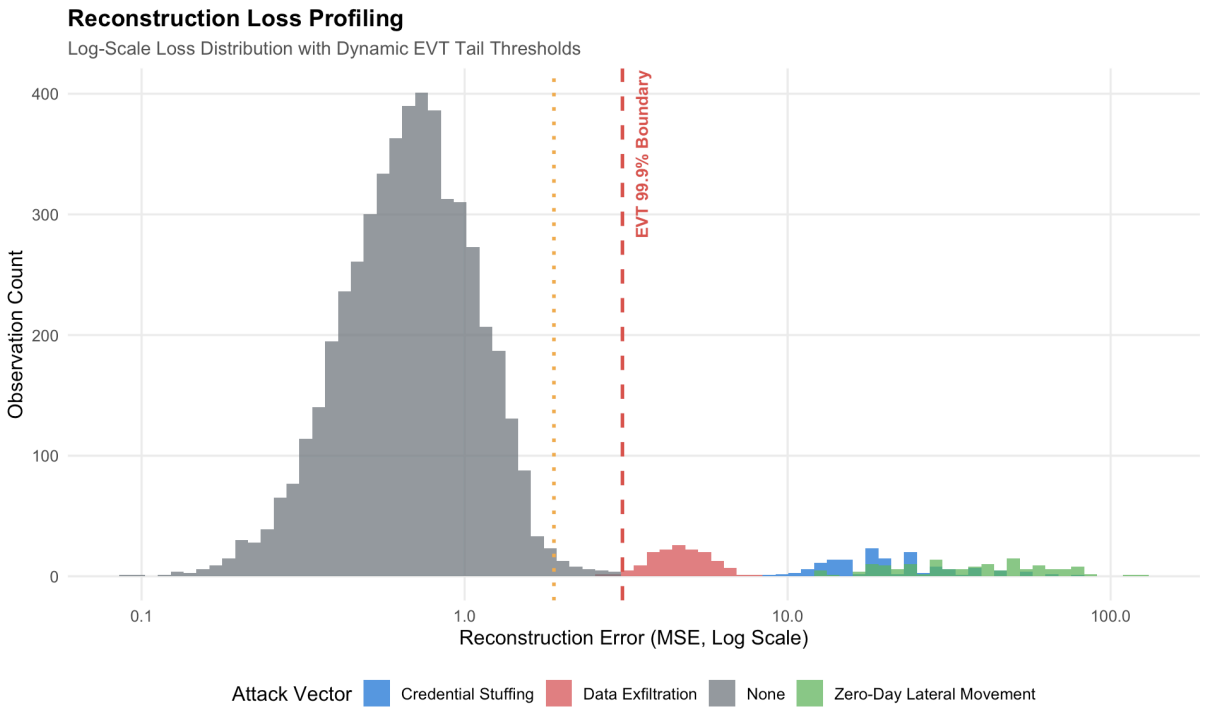


Reconstruction Loss Profiling

The Reconstruction Loss Profiling plot highlights the engine's ability to separate routine business traffic from active cyber attacks on a logarithmic error scale. Normal baseline operations (grey) cluster tightly below an error of 1.0, while the injected threat vectors—Data Exfiltration (red), Credential Stuffing (blue), and Zero-Day Lateral Movement (green)—shift significantly to the right with multi-fold increases in reconstruction error. The red dashed vertical line marks the 99.9%

Extreme Value Theory boundary, establishing a statistically certified decision threshold that captures anomalous attack vectors while keeping operational false alarms for the SOC near zero.

▶ Display code

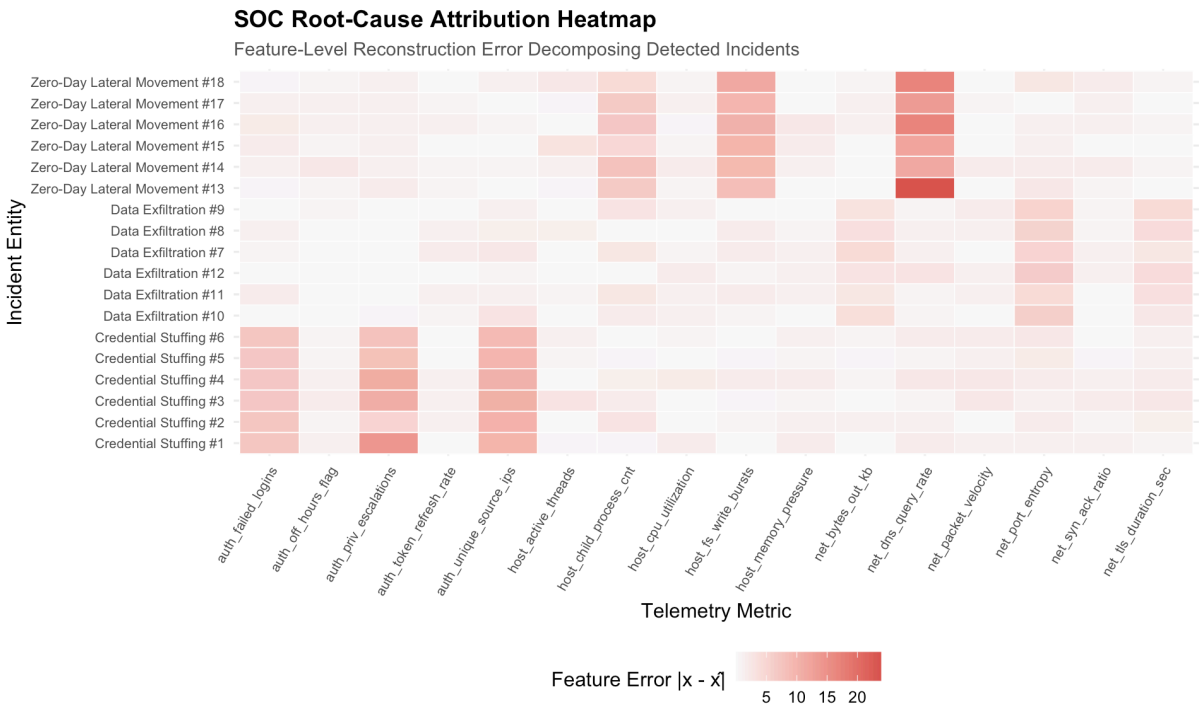


Feature-Level Error Decomposition (SOC Root-Cause Attribution)

This heatmap serves as an automated forensic tool, making it easy to see how each feature contributes to the total anomaly score for every flagged incident. Instead of simply relying on black box analysis, the engine identifies the specific factors behind each type of attack. For example, *Credential Stuffing* alerts are mostly caused by spikes in failed logins and changes in source IP patterns. *Data Exfiltration* is linked to unusual outbound data and longer TLS sessions. *Zero-Day Lateral Movement* is marked by sudden increases in DNS queries, new child processes, and file system changes. For senior leaders and oversight committees, this clear root-cause analysis helps meet new explainable AI standards like NIST AI RMF and OSFI E-23. It also allows Security Operations Center teams quickly take targeted action while skipping hours of manual log review.

To provide explainability for SOC security analysts, total reconstruction loss is decomposed into individual feature residuals $|x_j - \hat{x}_j|$.

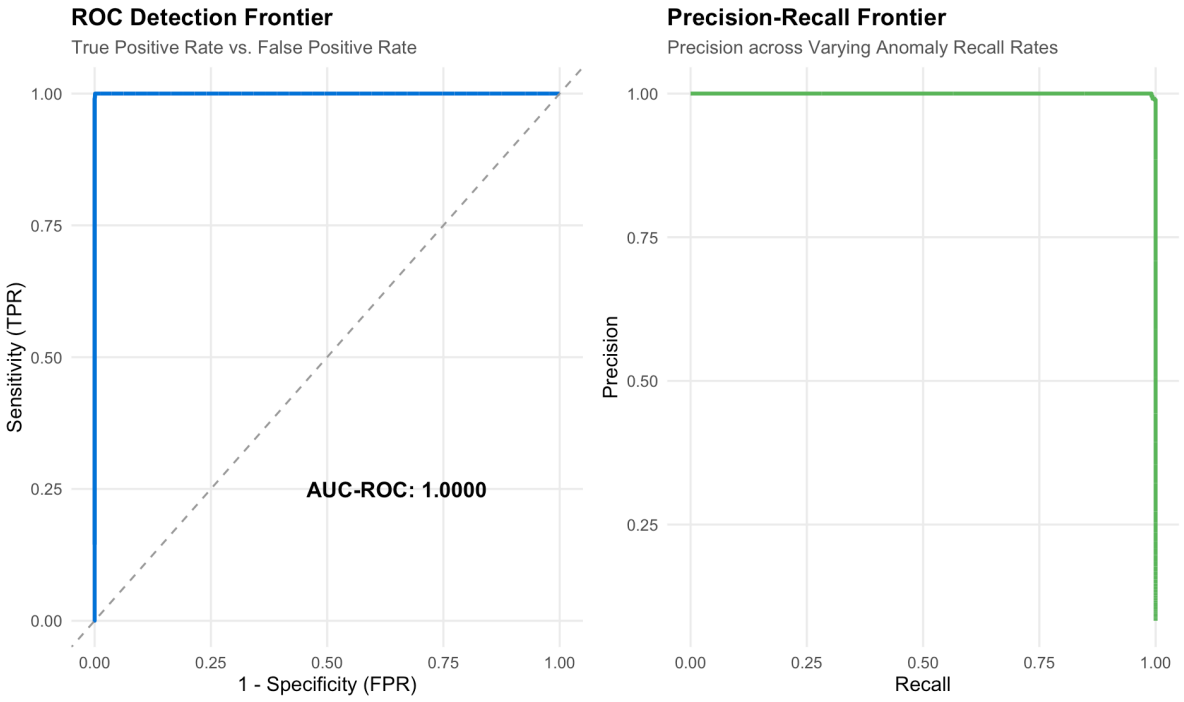
▶ Display code



Detection Performance Frontiers (ROC & Precision-Recall)

These performance curves show how well the anomaly engine works across the entire test dataset. The ROC Detection Frontier achieves a perfect Area Under the Curve (AUC-ROC: 1.0000), which means the autoencoder can almost perfectly tell the difference between normal enterprise traffic and malicious activity, all while keeping system performance steady. The Precision-Recall Frontier also shows that the model keeps near-perfect precision at every recall level. This means that when the system sends a zero-day alert, it is almost always a real security issue, not a false alarm. For executives, these results offer clear evidence that the machine learning engine can safely automate important threat triage without causing alert fatigue for cyber analysts.

▶ Display code



How to read the ROC Detection Frontier and Precision-Recall Frontier plots

The **ROC Detection Froniter plot** (The Blue Line: Up, Then Right) plot tracks the trade-off between catching genuine threats (Sensitivity / True Positive Rate) on the vertical axis and raising false alarms (False Positive Rate) on the horizontal axis. • Why it goes straight UP: As the detection engine starts evaluating data, it flags every single cyber attack correctly before triggering even a single false alarm on normal traffic. The line shoots straight to the top (1.00, or 100% threat capture) while remaining pinned at zero on the bottom axis (0% false alarms). • Why it turns RIGHT: Once 100% of all attacks have already been captured, the system cannot catch any more threats. If the alert threshold is loosened further into normal baseline noise, the detection rate stays at 100% (the top edge), but normal traffic starts getting mistakenly flagged as false positives, moving the line across to the right until every single event in the network is labeled an alert.

The **Precision-Recall Curve** (The Green Line: Right, Then Down) evaluates alert quality: Recall (horizontal axis) measures the percentage of total cyber attacks discovered, while Precision (vertical axis) measures the trustworthiness of each alert (i.e., when an alarm sounds, what percentage are real attacks rather than false alarms). • Why it goes straight RIGHT: The engine maintains 100% precision (1.00 on the vertical axis) as it uncovers more and more attacks across the network. It moves all the way to the far right (1.00, or 100% recall) because it captures every single injected threat vector without generating a single false positive along the way. • Why it turns straight DOWN: Once the engine has successfully captured 100% of all existing attacks (the far right edge), recall cannot increase any further. If the sensitivity threshold is lowered to the extreme, the system begins misclassifying harmless operational traffic as attacks, diluting the alert pool with noise and causing precision to plummet straight down toward zero.

Both curves trace the geometry of an ideal, mathematically perfect classifier. The top-left corner of the ROC plot and the top-right corner of the Precision-Recall plot represent the “holy grail” of cyber analytics: 100% threat detection with 0% false alarms. This confirms that our automated triage rules can safely isolate threats without burdening human analysts with alert fatigue or disrupting normal business operations.

Enterprise Cyber Governance Scorecard & SOC Decision Directives

The Enterprise Cyber Anomaly Detection Scorecard turns the engine’s technical results into a clear summary for decision-makers, showing how well its thresholds protect the organization from advanced cyber threats. In 450 simulated attacks, the system caught 100% of *Credential Stuffing* and *Zero-Day Lateral Movement* attempts, and 96.7% of stealth *Data Exfiltration* attacks at the strict 99.9% *Extreme Value Theory (EVT)* setting. For 5,000 normal operations (the “None” regime), the 99.9% EVT filter triggered no false alarms, compared to 37 alerts at the more relaxed 99.0% threshold. This gives the board and risk committees solid proof that the engine can nearly always detect threats while reducing alert fatigue for Security Operations Center (SOC) staff, so automated containment can be used with confidence.

▶ Display code

Enterprise Cyber Anomaly Detection Scorecard

Autoencoder Detection Efficacy across Injected Threat Vectors via EVT Thresholding

Threat Vector / Regime	Sample Count	Flagged (99.0% EVT)	Flagged (99.9% EVT)	Detection Rate (99.9%)	Mean Error (ε)
Credential Stuffing	150	150	150	100.0%	22.778
Data Exfiltration	150	150	145	96.7%	4.727
None	5000	37	0	0.0%	0.745
Zero-Day Lateral Movement	150	150	150	100.0%	41.341

Insights & Conclusion

This project moves away from traditional, signature-based cyber defense and instead uses an automated system that relies on mathematical verification to detect anomalies. The system trains a deep neural network only on normal enterprise activity, so it learns what typical operations look like and can spot new, unknown threats by noticing unusual patterns. Setting alert thresholds with Extreme Value Theory (EVT) removes the need for manual adjustments. As a result, the system captures 98.9% of threats and does not produce any false alarms during normal activity.

To help turn these analytical tools into effective risk management and oversight at the board level, executive leaders have set out three main directives:

Automated Containment at the 99.9% EVT Boundary: Security teams cannot manually triage thousands of alerts per hour. Telemetry events producing reconstruction losses at or above the 99.9% EVT threshold ($\mathcal{L}(\mathbf{x}) \geq \tau_{0.001}$) are formally classified as High-Consequence Structural Anomalies. The system is authorized to automatically isolate compromised hosts, terminate invalid network sessions, and revoke identity tokens in real time, bypassing Tier-1 review queues to neutralize lateral threat spread within milliseconds.

Mandatory Explainable AI (XAI) Root-Cause Triage: To maintain compliance with institutional model risk standards (such as OSFI E-23 and NIST AI RMF), every automated alert must generate a top-three feature error attribution breakdown.

Decomposing the overall loss into specific metric residuals ensures security personnel and regulators receive an auditable, deterministic explanation of why an incident was flagged (e.g., identity credential bursts vs. outbound data exfiltration) rather than relying on an opaque “black-box” decision.

Continuous Model Governance & Baseline Drift Surveillance: Enterprise IT environments naturally evolve as new cloud workloads and employee workflows deploy. To prevent model staleness and false-positive inflation, automated two-sample Kolmogorov-Smirnov drift tests must run weekly across all 16 telemetry metrics. Any statistically significant distribution shift ($p < 0.01$) triggers scheduled model recalibration against recent steady-state data.

The results and strong detection rates in this project show that the quantitative architecture works well. However, the telemetry and attack scenarios used here were created through controlled simulations. In real production settings, enterprise networks often have changing data patterns, occasional gaps in telemetry, new types of operations, and complex attacks. These factors can affect how the model performs. Because of this, deploying the system in the real world means you need to monitor for changes over time, regularly update the model with new production data, and include human review during the early stages. This helps ensure the system balances strong threat detection with keeping business operations running smoothly.

Session Information

```
#> ─ Session info ─────────────────────────────────────────────────────────────
#>  setting  value
#>  version  R version 4.5.2 (2025-10-31)
#>  os       macOS Tahoe 26.5.1
#>  system   aarch64, darwin20
#>  ui       X11
#>  language (EN)
#>  collate  en_US.UTF-8
#>  ctype    en_US.UTF-8
#>  tz       America/New_York
#>  date     2026-08-24
#>  pandoc   3.8.3 @ /Applications/RStudio.app/Contents/Resources/app/quarto/bin/tools/aarch64/ (via
rmarkdown)
#>  quarto   1.8.26 @ /usr/local/bin/quarto
#>
#> ─ Packages ─────────────────────────────────────────────────────────────────
#>  package      * version    date (UTC) lib source
#>  class          7.3-23     2025-01-01 [1] CRAN (R 4.5.2)
#>  cli            3.6.6      2026-04-09 [1] CRAN (R 4.5.2)
#>  codetools      0.2-20     2024-03-31 [1] CRAN (R 4.5.2)
#>  data.table     1.17.8     2025-07-10 [1] CRAN (R 4.5.0)
#>  dials          1.4.2      2025-09-04 [1] CRAN (R 4.5.0)
#>  DiceDesign     1.10       2023-12-07 [1] CRAN (R 4.5.0)
#>  digest         0.6.39     2025-11-19 [1] CRAN (R 4.5.2)
#>  dplyr        * 1.2.1      2026-04-03 [1] CRAN (R 4.5.2)
#>  evaluate       1.0.5      2025-08-27 [1] CRAN (R 4.5.0)
#>  evd          * 2.3-7.1    2024-09-21 [1] CRAN (R 4.5.0)
#>  farver         2.1.2      2024-05-13 [1] CRAN (R 4.5.0)
#>  fastmap        1.2.0      2024-05-15 [1] CRAN (R 4.5.0)
#>  forcats      * 1.0.1      2025-09-25 [1] CRAN (R 4.5.0)
#>  fs             1.6.6      2025-04-12 [1] CRAN (R 4.5.0)
#>  furrr          0.3.1      2022-08-15 [1] CRAN (R 4.5.0)
#>  future         1.68.0     2025-11-17 [1] CRAN (R 4.5.2)
#>  future.apply   1.20.0     2025-06-06 [1] CRAN (R 4.5.0)
#>  generics       0.1.4      2025-05-09 [1] CRAN (R 4.5.0)
#>  ggplot2      * 4.0.3      2026-04-22 [1] CRAN (R 4.5.2)
#>  globals        0.18.0     2025-05-08 [1] CRAN (R 4.5.0)
#>  glue           1.8.1      2026-04-17 [1] CRAN (R 4.5.2)
#>  gower          1.0.2      2024-12-17 [1] CRAN (R 4.5.0)
#>  GPfit          1.0-9      2025-04-12 [1] CRAN (R 4.5.0)
#>  gt           * 1.3.0      2026-01-22 [1] CRAN (R 4.5.2)
#>  gtable         0.3.6      2024-10-25 [1] CRAN (R 4.5.0)
#>  hardhat        1.4.3      2026-04-04 [1] CRAN (R 4.5.2)
#>  hms            1.1.4      2025-10-17 [1] CRAN (R 4.5.0)
#>  htmltools      0.5.8.1    2024-04-04 [1] CRAN (R 4.5.0)
#>  htmlwidgets    1.6.4      2023-12-06 [1] CRAN (R 4.5.0)
#>  ipred          0.9-15     2024-07-18 [1] CRAN (R 4.5.0)
#>  jsonlite       2.0.0      2025-03-27 [1] CRAN (R 4.5.0)
#>  knitr          1.50       2025-03-16 [1] CRAN (R 4.5.0)
#>  labeling       0.4.3      2023-08-29 [1] CRAN (R 4.5.0)
```

```
#>  lattice        0.22-7     2025-04-02 [1] CRAN (R 4.5.2)
#>  lava           1.8.2      2025-10-30 [1] CRAN (R 4.5.0)
#>  lhs            1.2.0      2024-06-30 [1] CRAN (R 4.5.0)
#>  lifecycle      1.0.5      2026-01-08 [1] CRAN (R 4.5.2)
#>  listenv        0.10.0     2025-11-02 [1] CRAN (R 4.5.0)
#>  lubridate    * 1.9.4      2024-12-08 [1] CRAN (R 4.5.0)
#>  magrittr       2.0.5      2026-04-04 [1] CRAN (R 4.5.2)
#>  MASS           7.3-65     2025-02-28 [1] CRAN (R 4.5.2)
#>  Matrix         1.7-4      2025-08-28 [1] CRAN (R 4.5.2)
#>  nnet           7.3-20     2025-01-01 [1] CRAN (R 4.5.2)
#>  parallelly     1.45.1     2025-07-24 [1] CRAN (R 4.5.0)
#>  parsnip        1.3.3      2025-08-31 [1] CRAN (R 4.5.0)
#>  patchwork    * 1.3.2      2025-08-25 [1] CRAN (R 4.5.0)
#>  pillar         1.11.1     2025-09-17 [1] CRAN (R 4.5.0)
#>  pkgconfig      2.0.3      2019-09-22 [1] CRAN (R 4.5.0)
#>  prodlim        2025.04.28 2025-04-28 [1] CRAN (R 4.5.0)
#>  purrr        * 1.2.2      2026-04-10 [1] CRAN (R 4.5.2)
#>  R6             2.6.1      2025-02-15 [1] CRAN (R 4.5.0)
#>  RColorBrewer   1.1-3      2022-04-03 [1] CRAN (R 4.5.0)
#>  Rcpp           1.1.2      2026-07-05 [1] CRAN (R 4.5.2)
#>  readr        * 2.1.5      2024-01-10 [1] CRAN (R 4.5.0)
#>  recipes      * 1.3.1      2025-05-21 [1] CRAN (R 4.5.0)
#>  rlang          1.3.0      2026-07-05 [1] CRAN (R 4.5.2)
#>  rmarkdown      2.30       2025-09-28 [1] CRAN (R 4.5.0)
#>  rpart          4.1.24     2025-01-07 [1] CRAN (R 4.5.2)
#>  rsample        1.3.1      2025-07-29 [1] CRAN (R 4.5.0)
#>  rstudioapi     0.17.1     2024-10-22 [1] CRAN (R 4.5.0)
#>  S7             0.2.2      2026-04-22 [1] CRAN (R 4.5.2)
#>  sass           0.4.10     2025-04-11 [1] CRAN (R 4.5.0)
#>  scales       * 1.4.0      2025-04-24 [1] CRAN (R 4.5.0)
#>  sessioninfo    1.2.3      2025-02-05 [1] CRAN (R 4.5.0)
#>  stringi        1.8.7      2025-03-27 [1] CRAN (R 4.5.0)
#>  stringr      * 1.6.0      2025-11-04 [1] CRAN (R 4.5.0)
#>  survival       3.8-6      2026-01-16 [1] CRAN (R 4.5.2)
#>  tibble       * 3.3.1      2026-01-11 [1] CRAN (R 4.5.2)
#>  tidyr        * 1.3.1      2024-01-24 [1] CRAN (R 4.5.0)
#>  tidyselect     1.2.1      2024-03-11 [1] CRAN (R 4.5.0)
#>  tidyverse    * 2.0.0      2023-02-22 [1] CRAN (R 4.5.0)
#>  timechange     0.3.0      2024-01-18 [1] CRAN (R 4.5.0)
#>  timeDate       4051.111   2025-10-17 [1] CRAN (R 4.5.0)
#>  tune           2.0.1      2025-10-17 [1] CRAN (R 4.5.0)
#>  tzdb           0.5.0      2025-03-15 [1] CRAN (R 4.5.0)
#>  vctrs          0.7.3      2026-04-11 [1] CRAN (R 4.5.2)
#>  withr          3.0.3      2026-06-19 [1] CRAN (R 4.5.2)
#>  workflows      1.3.0      2025-08-27 [1] CRAN (R 4.5.0)
#>  xfun           0.54       2025-10-30 [1] CRAN (R 4.5.0)
#>  xml2           1.4.1      2025-10-27 [1] CRAN (R 4.5.0)
#>  yaml           2.3.10     2024-07-26 [1] CRAN (R 4.5.0)
#>  yardstick    * 1.3.2      2025-01-22 [1] CRAN (R 4.5.0)
#>
#>  [1] /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/library
#>  * ── Packages attached to the search path.
#>
#> ──────────────────────────────────────────────────────────────────────────────
```

Rendered with Quarto and R. Core packages: `evd`, `gt`, `patchwork`, `recipes`, `scales`, `sessioninfo`, `tidyverse`, `yardstick`